Field Hours

CSCE 629 Analysis of Algorithms

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 45 minutes

Models, recurrences, and divide-and-conquer

An algorithm is a procedure plus a cost model. The model that this course uses counts comparisons, arithmetic, and pointer hops, and treats them as unit cost unless the problem is about bit operations. That model is wrong for a machine with a slow memory, and it is the right first model because it lets two designs be compared before anyone writes code. Always name the model. A bound with no model is a slogan.

Divide-and-conquer splits the input, solves the pieces, and pays a combine cost. Merge sort on n items obeys T(n) = 2T(n/2) + n, with T(1) = 1, because each level of the recursion does n merge steps and there are log base 2 of n levels. The closed form is Theta of n log n. The same recurrence shows up whenever you split in half and do linear work to stitch the halves.

The master theorem classifies T(n) = a T(n/b) + f(n). Let c be log base b of a. If f is polynomially smaller than n to the c, the leaves dominate and the bound is Theta of n to the c. If f is Theta of n to the c, the bound gains a log factor. If f is polynomially larger and a regularity condition holds, the root cost dominates. Apply it only when the split is equal and f is a simple polynomial.

Probabilistic algorithms belong in the same toolkit. Randomized quicksort picks the pivot at random. The expected number of comparisons is about 1.4 n log n even though one unlucky pivot sequence is quadratic. The expectation is over the coins, not over a promise that the input is random. That distinction is the whole point of a probabilistic algorithm: the input can be hostile, the coins are yours.

Where the cost of merge sort sits. Each band is one level and costs n.

Level 1: one merge, cost n

Level 2: two merges, cost n

Level 3: four merges, cost n

Leaves: n sorted singles

Merge sort on eight numbers

1. Input 5, 1, 8, 3, 7, 2, 6, 4. The top merge costs 8 comparisons in the worst case.

2. Two halves of four cost 4 each, and four pairs cost 2 each. Every level costs 8.

3. There are 3 levels because log base 2 of 8 is 3. Total comparisons are at most 24, which is 8 times log base 2 of 8.

Problem 1. Solve T(n) = 4 T(n/2) + n. Give the Theta bound and name the master-theorem case.

Answer. a = 4, b = 2, so log base b of a is 2. f(n) = n is O of n to the 2 minus epsilon, which is case 1.

The bound is Theta of n squared.

Problem 2. Randomized quicksort on n distinct keys uses expected comparisons under 2 n ln n. For n = 4, is a worst-case pivot sequence possible, and does that break the expectation claim?

Answer. Yes, a bad sequence exists: always pivot on the extreme key and the cost is quadratic on that path. The expectation averages over pivot coins, so a rare bad path does not cancel the claim. For n = 4 the bad paths are few and the average stays linearithmic.

Lecture 2 · 45 minutes

Graphs: search, shortest paths, spanning trees

A graph is vertices plus edges. Breadth-first search from a source visits each vertex and edge a constant number of times, so it is Theta of vertices plus edges. On an unweighted graph the first time you reach a vertex is a shortest path, measured in hops. Depth-first search is the same cost and answers a different question: finishing times, cycles, and the structure of the components.

Dijkstra's algorithm is breadth-first search with a priority on distance, and it requires nonnegative edge weights. With a binary heap the cost is O of (vertices plus edges) times log vertices. The invariant is that every vertex taken out of the queue already has its final distance. A negative edge breaks the invariant, because a later path could come back cheaper. If you have negative edges and no negative cycle, use Bellman-Ford, which is O of vertices times edges.

A minimum spanning tree connects every vertex at least total edge weight. Kruskal's algorithm sorts edges and adds one if it does not close a cycle. Prim's algorithm grows one tree, always adding the cheapest edge that leaves the tree. Both are correct because of the cut property: for any cut, the lightest edge across that cut is safe to include in some minimum tree. That one sentence is the proof you should be able to say out loud.

Model the graph before you pick the algorithm. An adjacency matrix is the right structure when the graph is dense, because a scan of a row is cheap and simple. An adjacency list is the right structure when the graph is sparse, which is most networks. A shortest-path code that is Theta of vertices squared is fine for a few hundred vertices and absurd for a million.

Dijkstra from A. Distances become final in this order.

1. Start A at distance 0
2. Settle B, distance 1
3. Settle C via B, distance 3
4. Settle D via C, distance 4

A four-vertex shortest path

1. Edges: A to B weight 1, A to C weight 4, B to C weight 2, B to D weight 6, C to D weight 1.
2. After settling A, the queue holds B at 1 and C at 4. Settle B next.
3. Relaxing B improves C from 4 to 3 and offers D at 7. Settle C at 3.
4. Relaxing C improves D from 7 to 4. The path is A, B, C, D and the distance is 4.

Problem 1. Same graph, but change C to D into a weight of -2. Why must you not run Dijkstra, and what is the correct distance from A to D?

Answer. A negative edge can make a settled distance wrong later. The path A, B, C, D now costs 1 + 2 + (-2) = 1. Dijkstra may already have settled D at 7 via B and will not repair it. Bellman-Ford will, because it relaxes every edge once per vertex.

Problem 2. Edges with weights: AB 3, AC 1, BC 1, CD 4, BD 5. Run the cut argument. What is a minimum spanning tree and its weight?

Answer. Lightest edge is AC at 1. Then BC at 1 is safe. Then AB would cycle, so skip it. CD at 4 is lighter than BD at 5. Tree A-C, C-B, C-D, total weight 6.

Lecture 3 · 40 minutes

Greedy choice and exchange

A greedy algorithm commits to a local choice and never reopens it. It is correct only when you can prove that some optimal solution makes that same choice. The proof is usually an exchange: take an optimal solution that disagrees with you, swap in your choice, and show the cost does not get worse. If you cannot write the exchange, you do not have a greedy algorithm. You have a guess.

Interval scheduling is the clean example. Meetings have a start and a finish. Selecting the meeting that finishes first, then repeating on the rest, maximizes the number of meetings. The exchange is: an optimal schedule's first meeting finishes no earlier than yours, so you can replace it with yours and the rest of the schedule still fits. Earliest start time does not have this property. A long meeting that starts first can block three short ones.

Huffman coding is greedy on frequencies. Repeatedly join the two lightest nodes. The exchange says the two rarest symbols can be siblings at the deepest level of some optimal tree. Prefix codes are not unique, so the tree you build is one optimal tree, not the only drawing. The weighted path length is what is optimal.

Greedy fails in public on the knapsack problem when items cannot be split. Taking the highest value-per-weight item first can fill the sack with something that blocks two better items. That failure is useful. It tells you the problem wants dynamic programming, which is the next lecture. Do not force a greedy story onto a problem that has overlapping substructure and a capacity constraint.

Same afternoon, two rules. Only one rule has an exchange proof.

Earliest finish

Take the 1 to 2 slot

Then the 2 to 4 slot

Three meetings fit

Earliest start

Take the 1 to 6 slot

It starts first

One meeting fits

Four meetings

1. Meetings: A 1-6, B 1-2, C 2-4, D 5-6. Times are hours on a clock.

2. Earliest finish picks B, then C, then D. Three meetings.

3. Earliest start picks A and stops. One meeting. The greedy rule is not 'be greedy'. It is the specific rule you can exchange.

Problem 1. Meetings 0-3, 1-2, 2-5, 4-6, 5-8. Touching endpoints is allowed. How many does earliest-finish accept, and which ones?

Answer. Sort by finish time. The earliest finish is 1-2. Next is 2-5, which starts when 1-2 ends. Next is 5-8, which starts when 2-5 ends. 0-3 loses to 1-2, and 4-6 overlaps 2-5. Three meetings: 1-2, 2-5, and 5-8.

Problem 2. Items for a 5-pound sack, no fractions: weight 4 value 6, weight 3 value 5, weight 2 value 3. Why does value-per-pound greedy fail?

Answer. Value per pound: 6/4 = 1.5, 5/3 about 1.67, 3/2 = 1.5. Greedy takes the 3-pound item first, then cannot take the 4-pound item, and may take nothing else. Value 5. Taking the 2-pound and the 3-pound items yields value 8. The local ratio was not safe.

Lecture 4 · 50 minutes

Dynamic programming

Dynamic programming is recursion plus a table, used when subproblems overlap and an optimal solution contains optimal solutions of its pieces. If the pieces do not overlap, divide-and-conquer is enough and the table is waste. If a local choice cannot be exchanged into an optimum, greedy is wrong and the table is the method. State the subproblem in one sentence before you write a loop. The sentence is the algorithm.

For the zero-one knapsack, the sentence is: best value using the first i items with capacity w left. The recurrence looks at item i and either skips it, or takes it when w is at least its weight and adds its value to the smaller capacity. Each cell is a maximum of two already-computed cells. With n items and capacity W the table has n times W cells, so the cost is Theta of n W. That is polynomial in W, not in the number of bits of W. If W is huge, this is not a free lunch.

Edit distance is the same shape on strings. The cell for prefixes of length i and j is the cheapest of three moves: delete a character, insert a character, or replace if the characters differ. Neighboring cells overlap so thoroughly that a recursive implementation without a table repeats the same prefix pair an exponential number of times. The table is n times m.

Write the reconstruction, not only the number. Store a parent pointer or recompute the argmax when you walk backward from the last cell. A distance with no alignment, or a knapsack value with no list of items, fails the problem you were actually asked. The walk is linear in the table's side length.

Each knapsack cell reads a skip or a take. Fill left to right, item by item.

Skip item -> Or take it -> Write max -> Next weight

Knapsack, capacity 5

1. Items in order: weight 2 value 3, weight 3 value 4, weight 4 value 5.

2. After the first item, capacity 2 through 5 holds value 3.

3. The second item at capacity 5 takes both the first and the second: value 3 + 4 = 7, weight 5.

4. The third item at capacity 5 is value 5, which loses to 7. Optimum is 7, items one and two.

Problem 1. Add a fourth item, weight 1 value 2, still capacity 5, using the optimum 7 from items one and two. What is the new optimum?

Answer. Weight 1 fits beside items one and two? Their weight is already 5, so no. Beside item two only: weights 3+1=4, values 4+2=6, worse than 7. Beside item one: weights 2+1=3, values 3+2=5. Beside item three: weights 4+1=5, values 5+2=7, a tie. The optimum stays 7.

Problem 2. Edit distance, insert and delete and replace each cost 1. Transform 'cat' into 'cut'. What is the distance and one alignment?

Answer. One replacement, a to u. Distance 1. The table agrees: matching c, then replace a, then matching t.

Lecture 5 · 45 minutes

NP-completeness and approximation

P is the set of decision problems a deterministic machine can answer in time polynomial in the length of the input. NP is the set where a proposed witness can be checked in polynomial time. Every problem in P is in NP, because the checker can ignore the witness and solve the problem. The useful question is the other direction, and this course treats it as open. You will not be asked to settle it. You will be asked to use it.

A problem is NP-complete when it is in NP and every problem in NP reduces to it by a polynomial-time mapping. The mapping must preserve yes and no. A reduction from a known NP-complete problem, usually 3-SAT or vertex cover or clique, is the ordinary way in. If you only reduce your problem to a known hard problem, you have shown nothing: easy problems also reduce to hard ones.

Approximation is what you do after the reduction succeeds and you still need an answer. For vertex cover, take any maximal matching and keep both endpoints of each matched edge. The cover has twice as many vertices as the matching. Some cover must pick at least one endpoint of each matched edge, so the optimum is at least the number of matched edges. The algorithm is a 2-approximation. It is not the cover a human would draw, and the factor 2 is proved.

A probabilistic algorithm is a different compromise. A Monte Carlo algorithm may be wrong with small probability and is usually cheap to repeat. A Las Vegas algorithm is always correct and its running time is the random variable, as with randomized quicksort. Do not call a heuristic an approximation unless you can name the worst-case factor. Do not call it Monte Carlo unless you can name the error probability.

Two honest ways to live with an NP-complete problem. A heuristic is neither, until you prove something.

Approximation

Always a legal answer

Cost within a factor

Vertex cover: factor 2

Monte Carlo

Sometimes wrong

Error you can bound

Repeat to drive it down

Vertex cover from a matching

1. Edges: a-b, c-d, and nothing else. A maximal matching is both edges.

2. The algorithm returns a, b, c, and d. Cover size 4.

3. The optimum is 2, for example a and c. The ratio is 4 over 2, which meets the factor and does not beat it.

Problem 1. You reduce sorting to vertex cover by a polynomial mapping. Does that prove sorting is NP-complete?

Answer. No. A reduction from A to B says B is at least as hard as A. Sorting is easy, so reducing it to vertex cover only shows you can encode an easy problem inside a hard one. Completeness needs a reduction from a known NP-complete problem, plus membership in NP.

Problem 2. A graph is a triangle, three edges. What cover does the matching algorithm return, and what is the optimum?

Answer. A maximal matching has one edge. The algorithm returns both endpoints, size 2. The optimum is also 2, because one vertex leaves an edge uncovered. Ratio 1 on this graph. The worst-case factor 2 is about other graphs, not this one.

Lecture 6 · 180 minutes

Amortized analysis

A worst-case bound on one operation can be the wrong way to charge a sequence. Amortized analysis bounds the average cost of an operation over a long run, and the bound holds on every sequence, not only on a random one. There are three standard accountings. Aggregate analysis finds the total cost of n operations and divides by n. The accounting method charges some operations more than they cost and stores the surplus as credit. The potential method stores that surplus in a function of the data structure. All three must give a number you can defend. 'Usually cheap' is not an analysis.

A dynamic array starts with capacity 1 and doubles when an insertion would overflow. The expensive insertions copy 1, then 2, then 4, then 8 cells, and so on. Across n insertions the copies sum to less than 2n, and the ordinary writes are n, so the total is under 3n. The amortized cost of one insertion is constant. A single insertion can still cost n. The claim is about the sequence.

A binary counter supports increment. The k-th bit flips every 2^k increments. In n increments the total flips are under 2n, so the amortized cost is 2. With a potential equal to the number of 1-bits, an increment that flips t trailing ones and sets one zero has actual cost t + 1 and potential change 1 - t. Amortized cost is actual plus

potential change, which is 2. The potential starts at 0 and stays nonnegative, so the sum of amortized costs upper-bounds the real cost.

Use amortization when a data structure is allowed to be expensive rarely and you can prove the rare events are paid for by the cheap ones. Do not use it to hide a quadratic algorithm that is quadratic on the sequence you actually run. State the potential, show it never goes negative, and show the amortized cost.

Copies when a doubling array grows to hold 8 items.

Insert into capacity 1: copy 1

Grow to 2: copy 1

Grow to 4: copy 2

Grow to 8: copy 4

Eight insertions into a doubling array

1. Capacities visited are 1, 2, 4, 8. Copy costs are 1 + 1 + 2 + 4 = 8.

2. Eight insertions also write 8 new cells. Total work is 16, which is 2 per insertion.

3. The last insertion, the one that grows 4 to 8, costs 4 copies by itself. The amortized bill is still constant because the earlier insertions were charged for that growth.

Problem 1. A stack supports push, pop, and multipop(k), which pops min(k, size) items. Each push or pop is one unit. What is the amortized cost of any of these operations over a sequence that starts from an empty stack?

Answer. Each item is pushed once and popped once, so n operations cost at most n pushes plus n pops. Aggregate cost is O(n), and the amortized cost of each operation is O(1). Multipop looks expensive and is paid for by earlier pushes.

Problem 2. Potential is the number of items in a binary heap. Does that potential prove that insert is amortized O(1)?

Answer. No. Insert already walks O(log n) levels, and this potential rises by 1, so the amortized cost is still O(log n). A potential only helps when it falls on the expensive operations. This one does not pay for the heap walk.

Lecture 7 · 180 minutes

Disjoint sets

A disjoint-set structure keeps a partition of elements. Find(x) returns the representative of the set containing x. Union(x, y) merges the two sets. The naive version is a parent pointer and a walk to the root. Without care, a union sequence builds a linked list and find costs linear time.

Union by rank attaches the shorter tree under the taller one, and the rank increases only when the ranks were equal. Any tree of rank r then has at least 2^r nodes, so the rank is at most log n and find is O(log n) even before compression.

Path compression makes every node on the find path point directly at the root. Combined with union by rank, the amortized cost of m operations on n elements is O(m alpha(n)), where alpha is the inverse of Ackermann's function. For every n you will ever store, alpha(n) is at most 4. The bound is not constant in the mathematical sense, and treating it as a small constant is the right engineering reading.

This structure is why Kruskal's algorithm is not 'sort plus a quadratic scan.' After the edges are sorted, each edge does two finds and maybe a union. On a connected graph that is the spanning-tree work, and it is dominated by the sort.

Two finds and a union on four elements.

1. find(a) walks to the root

2. find(c) walks to its root

3. union links the shorter root under the taller

4. the next find compresses the path

Four elements, union by rank

1. Start with a, b, c, d as singletons, rank 0.

2. Union(a, b) and union(c, d). Both winners have rank 1.

3. Union of those two roots makes one tree of rank 2. A find from a leaf passes two edges, not four. Rank forbids the long list.

Problem 1. You union n singletons by always linking the root of rank 0 under a growing chain, and you do not use union by rank. What is the worst-case cost of the last find?

Answer. Theta(n). The structure is a list. Union by rank exists to forbid that shape. Path compression alone also repairs it over a sequence, but one find before any compression still walks the list.

Problem 2. Kruskal sorts m edges and then runs a disjoint-set operation per edge. Which term dominates on a sparse connected graph with m = O(n)?

Answer. The sort, which is O(n log n). The disjoint-set work is O(n alpha(n)), which is slower-growing than the sort. If you leave the edges unsorted, the algorithm is wrong, not merely slower.

Lecture 8 · 180 minutes

Maximum flow

A flow network is a directed graph with a source, a sink, and a nonnegative capacity on each edge. A flow assigns a value to each edge that does not exceed capacity, and at every vertex other than the source and sink the flow in equals the flow out. The value of the flow is the net flow leaving the source.

The residual graph has a forward edge for unused capacity and a backward edge for flow you can undo. Ford-Fulkerson repeatedly finds a path from source to sink in the residual graph and pushes the bottleneck residual capacity along it. When no residual path remains, the flow is maximum. The max-flow min-cut theorem says that value equals the capacity of a minimum source-side cut. The cut is the set of vertices still reachable from the source in the final residual graph.

If you pick residual paths badly, and capacities are integers that share a large factor, Ford-Fulkerson can take many augmentations. Edmonds-Karp always augments along a shortest residual path, measured in hops, and finishes in O(vertices times edges squared) augmentations' worth of search. Dinic and push-relabel are faster. You should know why the residual backward edge exists: without it, an unlucky first path cannot be repaired.

Integer capacities produce an integer maximum flow if you start from zero and augment by integer bottlenecks. That fact is what lets flow solve combinatorial problems. It is not true of an arbitrary linear program with the same picture.

One augmentation, then the residual update.

Find a residual path -> Push the bottleneck -> Add backward edges -> Repeat until no path

A four-edge network

1. Edges: source to A capacity 3, source to B capacity 2, A to sink capacity 2, B to sink capacity 3, A to B capacity 1. All start at flow 0.

2. Path source-A-sink pushes 2. Path source-B-sink pushes 2. Residual source-A still has 1, and A-B has 1, and B-sink has 1 left, so a third path pushes 1.

3. Total value is 5. The source's outgoing capacity is 5, so 5 is maximum. The backward edge A to source would let you undo the first path if a later path needed the capacity.

Problem 1. After a maximum flow, which vertices form the source side of a minimum cut?

Answer. The vertices reachable from the source in the residual graph, including the source. Every edge leaving that set is saturated, and the cut capacity equals the flow value.

Problem 2. Capacities are 1000000 on two parallel paths that share a middle edge of capacity 1, and you always augment one unit. What goes wrong with plain Ford-Fulkerson, and what does Edmonds-Karp change?

Answer. A poor choice of paths can augment one unit at a time and take a million steps. Edmonds-Karp picks a shortest residual path, so it cannot nibble forever. The shortest-path rule bounds the number of augmentations by a polynomial in the graph size, independent of the numeric capacities.

Lecture 9 · 180 minutes

Matching and cuts

A matching is a set of edges with no shared vertex. In a bipartite graph, maximum matching reduces to maximum flow. Put a source with an edge of capacity 1 to every left vertex, direct the original edges left to right with capacity 1, and put an edge of capacity 1 from every right vertex to the sink. An integer maximum flow uses each original edge 0 or 1 times, and the used edges are a matching. The flow value is the matching size.

König's theorem says that in a bipartite graph the size of a maximum matching equals the size of a minimum vertex cover. The min-cut of the flow network tells you the cover. This is the cleanest example of a problem that is polynomial because a theorem identifies it with flow, while the same sentence is false for general graphs.

On general graphs, Edmonds' matching algorithm is polynomial and more technical. You do not need the full

blossom implementation in this course. You do need to know that 'add an edge if it is free' is not optimal. A single augmenting path of length three already improves a greedy matching.

Project selection, image segmentation, and baseball elimination are also cuts. The pattern is the same: encode a yes-or-no choice as a finite capacity, and read the answer off a min cut. If you cannot name the capacity that represents the choice, you do not have a reduction yet.

Matching on the left, the flow network on the right.

People and jobs

A can do job 1 or 2

B can do job 2

C can do job 1 or 3

Unit capacities

Source feeds A, B, C at capacity 1

Jobs drain to the sink at capacity 1

An integer flow is a matching

Three people, three jobs

1. Edges: A-1, A-2, B-2, C-1, C-3.

2. A greedy pass might match A-2 and then C-1, size 2, leaving B unmatched because job 2 is taken.

3. An augmenting repair matches A-1, B-2, C-3. Size 3. The flow algorithm finds that repair because it can send flow backward along A-2 and undo the greedy choice.

Problem 1. Why are the capacities from the source equal to 1, not equal to the number of jobs a person can do?

Answer. Capacity 1 means the person is used once. If a person may take two jobs, that edge's capacity is 2, and the flow is no longer a matching. The capacity is the constraint you meant, not a default.

Problem 2. A bipartite graph has a matching of size 4 and 9 vertices. What is the size of a minimum vertex cover?

Answer. 4. König's theorem applies because the graph is bipartite. On a general graph the cover can be larger than the matching, and this shortcut is false.

Lecture 10 · 180 minutes

Linear programs

A linear program maximizes or minimizes a linear function over a polyhedron defined by linear inequalities. For algorithms, the useful facts are these. An optimum, if it exists and the feasible region is pointed, occurs at a vertex. A vertex is defined by a set of tight constraints. And every linear program has a dual whose value

upper-bounds the primal. If you exhibit a feasible primal and a feasible dual with the same value, both are optimal. That is strong duality, and it needs a constraint qualification that everyday nonnegative programs satisfy.

You can solve a small program by listing vertices. You do not list vertices on a program with hundreds of inequalities. The simplex method walks from vertex to vertex and is fast in practice and exponential on some crafted instances. Interior-point methods are polynomial. This course treats the solver as a tool whose contract you must state: it returns an optimal vertex, or it reports that the program is infeasible or unbounded.

An integrality gap is the difference between the best fractional solution and the best integer solution. Maximum matching on bipartite graphs has no gap: the flow polyhedron is integral. The naive linear program for vertex cover, with x_v between 0 and 1 and x_u + x_v at least 1 on every edge, can return halves. On a triangle the fractional optimum is 1.5 and the integer optimum is 2. Rounding those halves gives a 2-approximation, which is the algorithm from the NP lecture seen from the other side.

Writing a problem as a linear program is not the same as solving it. If you need integer variables, you have an integer program, and the solver may be exponential. Say which one you wrote.

Primal choices and the dual bound.

Primal

Maximize x + y

x + 2y <= 4

2x + y <= 4

x, y >= 0

Dual

Minimize 4p + 4q

p + 2q >= 1

2p + q >= 1

p, q >= 0

Two variables, two inequalities

1. The binding point of x + 2y = 4 and 2x + y = 4 is x = 4/3, y = 4/3. The objective is 8/3.

2. The other vertices are (0, 0) value 0, (2, 0) value 2, and (0, 2) value 2. The best vertex is 8/3.

3. Dual values p = 1/3, q = 1/3 are feasible: p + 2q = 1 and 2p + q = 1. Dual objective 4/3 + 4/3 = 8/3. Equal values, so both are optimal.

Problem 1. On a triangle, the vertex-cover linear program sets every variable to 1/2. Why is that feasible, and why is it not a vertex cover?

Answer. Each edge sums two halves and gets 1, so the inequalities hold. A vertex cover needs a 0-1 choice that actually touches every edge. Halves are not a cover. The integrality gap on this graph is 2 / 1.5.

Problem 2. You add the constraint that x is an integer. Is the program still a linear program, and does the vertex argument still give you the optimum by itself?

Answer. No. It is an integer linear program. The optimum may sit in the interior of an edge of the relaxation, and listing fractional vertices can miss it. You need a method that enforces integrality.

Lecture 11 · 180 minutes

Approximation

An approximation algorithm returns a feasible solution whose cost is within a proven factor of the optimum, on every input. A factor of 2 for a minimization problem means the cost you return is at most twice the optimum, not twice some other heuristic. You usually cannot compute the optimum, so the proof compares your cost to a lower bound: a relaxation, a greedy charging argument, or the size of a maximal structure.

The greedy set-cover algorithm repeatedly picks the set that covers the most still-uncovered elements. Its cost is at most H_k times the optimum, where k is the size of the largest set and H_k is the k-th harmonic number, about ln k. The proof charges each newly covered element the set's price divided by how many elements that pick covered, and shows that the optimum would have had to pay a comparable bill.

Metric traveling salesman, where distances satisfy the triangle inequality, has a 2-approximation: build a minimum spanning tree, walk around it, and shortcut repeated vertices. The tree is at most the optimum tour, the walk is twice the tree, and shortcutting does not lengthen a metric tour. Christofides' algorithm improves the factor to 1.5 by adding a matching on the odd-degree vertices. Drop the triangle inequality and the problem has no constant-factor approximation unless P equals NP.

Some problems have a polynomial approximation scheme: for any epsilon you can get within 1 + epsilon in time that may be exponential in 1/epsilon but polynomial in the input. The knapsack dynamic program from the earlier lecture becomes a scheme by rounding item values. A scheme is a stronger promise than a fixed factor, and it is not available for every NP-hard problem.

Charging in greedy set cover. Later elements pay more.

Pick covers 4 new elements, each charged cost/4

Pick covers 2, each charged cost/2

Last element charged the full set cost

Sum of charges is at most H_k times optimum

Universe of four elements

1. Elements 1, 2, 3, 4. Sets: {1, 2} at cost 2, {3} at cost 2, {4} at cost 2, and {1, 2, 3, 4} at cost 7.

2. Greedy first takes {1, 2}, because it covers the most per cost among the cheap sets. Then it takes {3} and {4}. Total cost 6.

3. The optimum is also those three sets, or the big set at cost 7. Greedy matched the optimum here. The harmonic bound says it might have been worse by about ln 4, not that it always is.

Problem 1. Why does the MST walk give a 2-approximation only when distances are metric?

Answer. Shortcutting a repeated vertex replaces a path by a direct edge. The triangle inequality says that edge is no longer than the path. Without it, the shortcut can be the expensive edge in the whole graph, and the tour you return can be arbitrarily worse than the optimum.

Problem 2. A maximization problem returns a solution of value 10, and you proved the optimum is at most 40. What approximation factor should you claim, and which direction does the factor point?

Answer. You achieved 1/4 of the optimum, so this is a 4-approximation in the 'optimum over algorithm' convention, or a 0.25-approximation in the ratio convention. Say which convention you are using. Do not call it a 4-approximation for a maximization problem without saying that larger would have been better.

Lecture 12 · 180 minutes

Randomized algorithms

A Monte Carlo algorithm always runs quickly and is correct with probability you can push upward by repetition. A Las Vegas algorithm always returns the right answer and has a random running time, as randomized quicksort does. Name which one you are using. 'Randomized' by itself does not say what fails when the coins are unlucky.

Karger's min-cut algorithm contracts a random edge, merging its endpoints, until two vertices remain. The edges between them are a cut. One trial finds a specific minimum cut with probability at least 2 over n times (n - 1), which is about 2/n^2. Repeat the trial n squared times and the failure probability drops below a constant. A tighter implementation repeats inside the recursion and gets a high probability of success in near-quadratic time.

Universal hashing says a random hash function from a small family collides a fixed pair of keys with probability 1 over the number of buckets. The adversary can pick the keys, but not after seeing the function. That is why a hash table with chaining has expected chain length O(1 + load) against any fixed set. If the adversary sees the function first, they can pile every key into one bucket, and the expectation claim is void.

A random sample is not a randomized algorithm unless the correctness probability is over your coins on the worst input. Shuffling the input and hoping is a different, weaker claim.

One Karger trial.

1. Pick a uniform remaining edge

2. Contract its endpoints

3. Stop at two supernodes

4. The edges between them are the cut

Four vertices, one trial

1. A 4-cycle has four edges, and the minimum cuts are the pairs of opposite edges, size 2. There are also worse cuts.

2. The probability a given min-cut survives the first contraction is 2/4, because two of the four edges are in that cut and must not be picked. The formula 2/(n(n-1)) at n = 4 is 2/12 = 1/6, which is a lower bound on surviving every contraction, not the first one alone.

3. One trial is not enough. Order n squared independent trials make missing every one unlikely. The algorithm never certifies that the cut it returned is minimum unless you compare values across trials and keep the smallest.

Problem 1. You hash n keys into n buckets with a hash function the user supplied. An attacker who can see the function inserts the keys. What is the worst chain length, and which hypothesis of universal hashing failed?

Answer. The attacker can put all n keys in one bucket, so the chain is n. Universal hashing needs the function to be chosen from the family after the keys are fixed, or at least hidden from the adversary. A public function is an ordinary array.

Problem 2. Randomized quicksort always returns a sorted array. Is it Monte Carlo or Las Vegas, and what does repetition buy you?

Answer. Las Vegas. The answer is always correct. Repetition does not change correctness. A new run only rerolls the running time. You stop when that run finishes.

Lecture 13 · 180 minutes

Polynomials and a fast product

Two polynomials of degree under n have a product of degree under 2n. The naive product multiplies every pair of coefficients and takes Theta(n squared) arithmetic operations. The fast product changes representation. Evaluate both polynomials at 2n points, multiply those numbers pointwise, and interpolate the unique polynomial of degree under 2n that passes through the products. Evaluation and interpolation at arbitrary points are still quadratic. At the complex roots of unity they are the discrete Fourier transform, which factors into two half-size transforms and runs in Theta(n log n).

You do not need the complex numbers to believe the representation claim. A polynomial of degree under m is determined by its values at any m distinct points. That is Lagrange interpolation, and it is the reason the pointwise product determines the coefficient product. The Fourier transform is only the fast way to enter and leave that representation.

The same idea multiplies large integers, because an integer is a polynomial evaluated at the base. It builds circulant convolutions for signal processing. And it is the rare divide-and-conquer recurrence, T(n) = 2 T(n/2) + Theta(n), whose extra linear work is the butterfly, not a merge of records. The master theorem's tie case gives Theta(n log n).

Exact arithmetic matters. The roots of unity are not all integers. A number-theoretic transform uses a modulus and a root of unity in that modulus when you need exact integer coefficients. Floating-point FFT output needs rounding. Say which one your implementation does before you compare it to the naive product.

Multiply by changing representation.

Coefficients -> Values at 2n points -> Pointwise products -> Interpolate coefficients

A product you can check by hand

1. (1 + 2x) times (3 + 4x).

2. Coefficient way: 1 times 3 is 3, 1 times 4x plus 2x times 3 is 10x, 2x times 4x is 8 x squared.

Product: 3 + 10x + 8 x squared.

3. Value way, at x = 0, 1, and 2: the inputs are (1, 3), (3, 7), and (5, 11). Products are 3, 21, and 55. The unique quadratic through those points is the same 3 + 10x + 8 x squared. Three points determine it. The FFT would use four roots of unity for a degree bound of 4, including the zero x cubed coefficient.

Problem 1. Naive multiplication of two degree-999 polynomials does about a million coefficient products. The FFT method is Theta(n log n) on n = 2000 points. Which term of the master recurrence is the 'log n' coming from?

Answer. T(n) = 2 T(n/2) + Theta(n). The work per level is linear and there are log n levels, so the log comes from the depth of the recursion, not from a sort. The leaves are the coefficient pairs after the transforms, and they do not dominate beyond that bound.

Problem 2. You evaluate both polynomials at n points and interpolate. Why is that not enough when each input already has degree n - 1?

Answer. The product has degree up to 2n - 2, so it needs 2n - 1 distinct points to be determined. n points leave a whole space of polynomials that agree on those points and disagree elsewhere. You would reconstruct the wrong product.

Lecture 14 · 180 minutes

A proof you can hand in

The last week is the habit the earlier weeks were for. Given a new problem, you either give a polynomial algorithm with a named invariant, or you prove it NP-complete and then give an approximation or an exponential algorithm whose base you can state. A paragraph that only says the problem 'seems hard' is not a solution.

A reduction has three lines. What is the transformation, why is it polynomial, and why does a yes-instance map to a yes-instance in both directions. The reverse direction is the one people skip, and it is the one that makes the proof fail. If you add extra vertices, say what they do to the answer.

Independent set and vertex cover are the clean pair. A set of vertices is independent if and only if its complement is a vertex cover, because an edge is either inside the set, which forbids independence, or it meets the complement. So the graph has an independent set of size k if and only if it has a vertex cover of size n - k. The reduction is the identity on the graph plus a change of the number. It is polynomial. Both directions are the same sentence.

Write the final problem of this course as if someone else will implement it. Name the model, the bound, the invariant, and the case that breaks a naive version. That write-up is the exam.

The same graph, two questions.

Independent set

No edge inside the set

Size k

Complement may be large

Vertex cover

Every edge touches the set

Size n - k

Same decision, different number

A triangle plus a pendant

1. Vertices 1, 2, 3 form a triangle. Vertex 4 connects only to 3. n = 4.

2. An independent set of size 2: {1, 4}. The complement {2, 3} is a vertex cover. Check the pendant edge 3-4: it touches 3. Check the triangle: every edge meets {2, 3}.

3. {1, 2} is not independent, and its complement {3, 4} is not a vertex cover, because edge 1-2 touches neither. The equivalence did not break. The set was simply not feasible on either side.

Problem 1. You reduce independent set to vertex cover by outputting the same graph and the same number k. Is the reduction correct?

Answer. No. The right number on the cover side is n - k. Keeping k maps a large independent set to a question about a small cover, which is a different question. Both directions fail on the triangle, which has an independent set of size 1 and no vertex cover of size 1.

Problem 2. Give a polynomial algorithm or a one-sentence hardness reason for this problem: given a list of intervals, pick the largest subset of non-overlapping intervals.

Answer. Polynomial. Sort by finishing time and take the next interval that starts after the previous pick ends. The exchange argument from the greedy lecture shows this is optimal. It is not a packing of arbitrary sets, which would be set packing and NP-hard. The interval structure is the whole algorithm.